

You Only Look Once: Unified, Real-Time Object Detection

Date	@September 12, 2025
Keyword	CNN Real-Time Object Detection Recurrent Networks
Status	Done
URL	https://arxiv.org/pdf/1506.02640

Overview

이전의 object detection을 수행하던 모델들은 classifier를 사용해 detection을 수행했으나 YOLO는 하나의 neural network 사용해 bounding box를 탐지하고 class probability를 추정한다.

- YOLO는 실시간으로 1초에 45 frame 처리가 가능하고 Fast YOLO는 초당 155 frame 처리가 가능하다.
- YOLO는 DPM이나 R-CNN과 같은 detection method들보다 좋은 성능을 보인다.
- YOLO는 localization error를 많이 만들지만 false positive 예측률이 낮다.
- YOLO는 객체 general representation을 학습한다.

Introduction

현재의 detection system은 classifier를 재활용해(원래 객체의 클래스를 식별하도록 훈련된 classifier) object를 detection한다.

- DPM(Deformable Parts Models) 같은 경우는 sliding window approach를 사용해 크기가 같은 location마다 classifier가 작동하고
- R-CNN은 region proposal method를 사용해 potential bounding box들을 생성하고 classifier에 넣어준다.

이후 post-processing을 통해 bounding box를 정제한다. → 중복 detection을 제거하고 box들을 rescore한다.

⇒ 이러한 복잡한 pipeline은 optimize하기 어렵고 느리다.

우리는 이러한 object detection task를 하나의 regression problem으로 간주하고 하나의 convolutional network를 사용해 image pixel로부터 바로 bounding box coordinate과 class probability를 예측한다.

- 이러한 점 때문에 full image에서 train되고 detection performance를 바로 optimize 할 수 있다.

이러한 YOLO의 장점으로...

1. 복잡한 pipeline을 사용하지 않기 때문에 매우 빠르다.

다른 real-time system보다 2배의 mean average precision을 달성했다.

2. Bounding box를 예측할 때 이미지의 전역적인 정보를 활용한다.

Training과 test time에 image 전체를 활용하기 때문에 object의 존재와 함께 object의 contextual information을 같이 학습하게된다.

반면에 R-CNN은 큰 context를 보지 못하기 때문에, background patch들을 잘못 예측 하는 경우가 잦다.

3. YOLO는 object의 generalizable representation을 학습한다.

새로운 domain이나 예상치 못한 input에 대해서도 다른 model들보다 잘 작동한다.

Methods: Unidied Detection

하나의 neural network를 사용해 object detection의 여러 component를 통합한다.

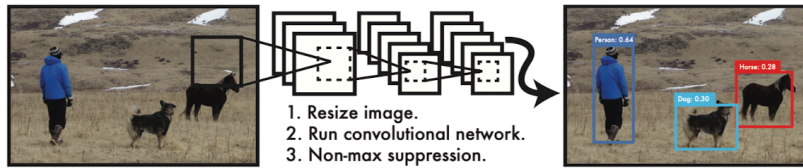


Figure 1: The YOLO Detection System. Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to 448×448 , (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model's confidence.

- 동시다발적으로 하나의 image에 존재하는 모든 bounding box와 그 class를 예측한다.
⇒ 하나의 object를 찾기 위해 이미지 전체의 정보를 사용한다.

YOLO의 핵심 로직

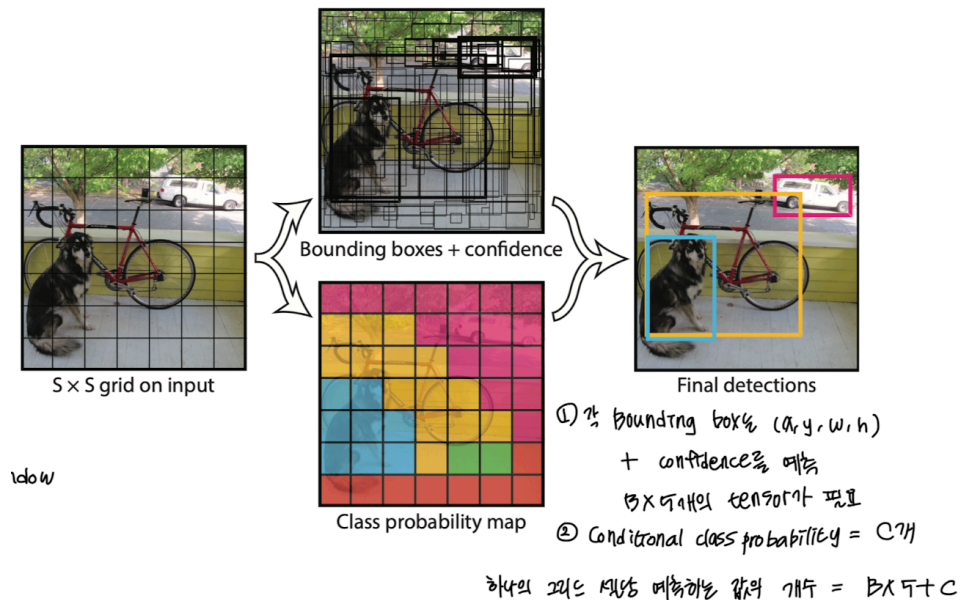


Figure 2: The Model. Our system models detection as a regression problem. It divides the image into an $S \times S$ grid and for each grid cell predicts B bounding boxes, confidence for those boxes, and C class probabilities. These predictions are encoded as an $S \times S \times (B * 5 + C)$ tensor.

1. input image를 $S \times S$ 의 grid로 나눈다.

object의 중심을 포함하는 grid cell이 object를 detecting하는데 책임을 진다. → 뒤에 나옴

2. 각 grid cell이 B개의 bounding box와 confidence score를 예측한다.

$$\text{Pr}(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}}$$

각 bounding box는 x, y, w, h (coordinate, width, height)와 C conditional class probability($\text{Pr}(\text{Class}_i | \text{Object})$)를 예측한다.

Test time에는 conditional class probability와 각 box의 confidence prediction을 곱한다.

$$\text{Pr}(\text{Class}_i | \text{Object}) * \text{Pr}(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}} = \text{Pr}(\text{Class}_i) * \text{IOU}_{\text{pred}}^{\text{truth}}$$

⇒ 이는 class-specific confidence score를 보여주는데 이는 object가 나타날 probability와 box가 object에 잘 fit하는지를 모두 표현한다.

Network Design

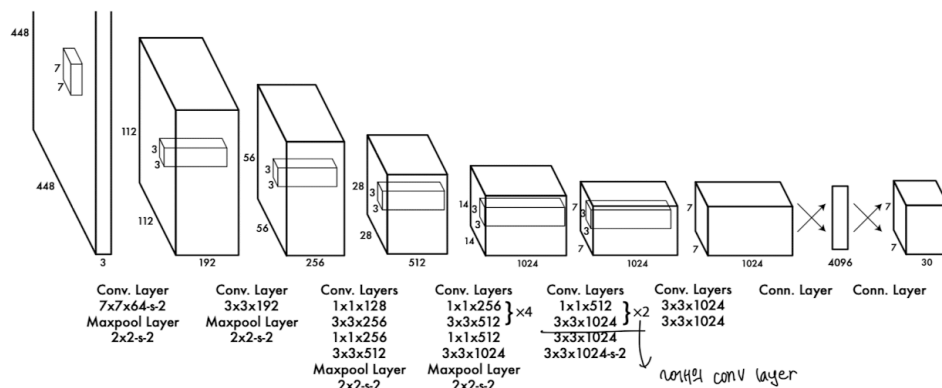


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

- Fast YOLO는 convolution layer의 개수와 filter의 개수가 비교적 적다.

구현 시에 GoogLeNet architecture를 따랐지만 inception module 대신에 1 x 1 convolution을 사용해 feature space를 줄인다.

Training

1. ImageNet 1000-class competition dataset을 사용해 convolutional layer를 pretrain한다.
 2. Model이 detection을 수행할 수 있도록 4개의 convolutional layer와 2개의 fully connected layer를 추가한다.
 3. 마지막 layer는 class probability와 bounding box coordinate을 모두 예측한다.
bounding box의 w 와 h 를 0과 1 사이로 정규화해 사용하였고, coordinate도 특정 grid cell을 기준의 offset으로 사용해 0과 1 사이의 값을 가지도록 했다.
- LeakyReLU를 사용
 - Sum-squared error를 사용
→ optimize하기 쉽기 때문에!

하지만 localization error와 classification error를 동등하게 취급하기 때문에 문제가 발생할 수 있다. + 대부분의 grid cell은 object를 가지고 있지 않는데 이는 confidence score를 0으로 만들려는 경향이 있고 그럴경우 객체가 있는 cell로부터 오는 gradient를 압도하게 된다.

⇒ $\lambda_{\text{coord}} = 5$ 와 $\lambda_{\text{noobj}} = .5$ 를 사용해 bounding box coordinate prediction의 loss의 영향력은 증가시키고 객체가 없는 cell의 confidence prediction의 loss의 영향력은 감소시킨다. ⇒ 위치 정확도 해결과 배경 셀의 과도한 영향력 문제 해결

YOLO는 하나의 grid cell에서 여러개의 bounding box를 예측하게되고 우리는 gt와 IOU가 가장 높은 cell을 선택해 object detection에 책임을 묻게 된다.

Multi-part loss function

$$\begin{aligned}
& \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbf{1}_{ij}^{\text{obj}} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] \\
& + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbf{1}_{ij}^{\text{obj}} \left[(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] \\
& + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbf{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\
& + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbf{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
& + \sum_{i=0}^{S^2} \mathbf{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2
\end{aligned}$$

1. Object가 존재하는 grid cell i 의 책임을 가지는 predictor j 에서 coordinate loss
2. Object가 존재하는 grid cell i 의 책임을 가지는 predictor j 에서 width와 height loss
3. Object가 존재하는 grid cell i 의 책임을 가지는 predictor j 에서 confidence-score loss
4. Object가 존재하지 않는 grid cell i 의 책임을 가지는 predictor j 에서 confidence-score loss
5. Object가 존재하는 grid cell i 에 대한 conditional class probability의 loss

여기서 핵심은 loss function은 object가 grid cell에 있을 경우, classification loss에 패널티를 주고, predictor가 gt box에 대한 책임이 있을 때 bounding box coordinate loss에 패널티를 준다.

Q. 어떤 항에는 λ 가 곱해지지 않는 이유?

A. 신뢰도나 클래스 확률은 위치 정확도를 강조하기 위해 도입된 λ_{coord} 와의 직접적인 관계가 존재하지 않음

객체가 있는 셀의 confidence-score loss는 배경 셀의 과도한 영향력 문제를 해결하기 위해 도입된 λ_{noobj} 와 직접적인 관계가 존재하지 않음

Inference

Training과 마찬가지로 test image에 대한 detection도 한번의 network evaluation이면 충분하다. → 매우 빠른 속도

- Multiple Detection 문제

Grid design은 bounding box prediction 시에 spatial diversity을 유도해 이 문제를 감소시켰지만

큰 object나 경계에 존재하는 object의 경우 여러개의 cell에서 localizing될 수 있기 때문에 Non-maximal suppression 알고리즘을 사용해서 multiple detection 문제를 해결한다.

Non-max suppression 알고리즘 작동 단계

1. 하나의 클래스에 대한 bounding boxes 목록에서 가장 높은 점수를 갖고 있는 bounding box를 선택하고 목록에서 제거합니다. 그리고 final box에 추가합니다.
2. 선택된 bounding box를 bounding boxes 목록에 있는 모든 bounding box와 IoU를 계산하여 비교합니다. IoU가 threshold보다 높으면 bounding boxes 목록에서 제거합니다.
3. bounding boxes 목록에 남아있는 bounding box에서 가장 높은 점수를 갖고 있는 것을 선택하고 목록에서 제거합니다. 그리고 final box에 추가합니다.
4. 다시 선택된 bounding box를 목록에 있는 box들과 IoU를 비교합니다. threshold보다 높으면 목록에서 제거합니다.
5. bounding boxes에 아무것도 남아 있지 않을 때 까지 반복합니다.
6. 각각의 클래스에 대해 위 과정을 반복합니다.

- 모델 훈련시에 작은 bounding box의 error와 큰 bounding box의 error를 똑같이 취급한다.

작은 bounding box는 더 작은 error를 만들지만 이는 IOU에 더 큰 영향을 미친다.

Comparison to Other Detection Systems

- **Deformable parts models**

Sliding window 방식을 사용해 object detection을 하는 모델

- feature 추출, region 분류, bounding box prediction에 모두 다른 파이프라인을 사용

- **R-CNN**

Region proposal method를 사용해 추출한 후보 region들에 대해 convolutional network와 SVM을 사용하는 방식

- 파이프라인의 부분부분이 독립적으로 튜닝되어양하고 inference가 매우 느리다.

- **Other Fast Detector**

Fast, Faster R-CNN이나 DPM의 속도를 향상시키기 위한 다양한 방법들

- **Deep MultiBox**

Region of interest를 예측하기 위해 convolutional neural network를 학습시키는 방법

- 더 커진 detection pipeline을 사용한다.

- **OverFeat**

Localization을 수행하기위해 convolutional neural network를 훈련하고 localizer를 사용해 detection을 수행하는 모델

- Global context를 보지 못한다는 단점이 있다.

- **MultiGrasp**

YOLO와 유사하지만 grasp detection은 하나의 graspable region을 찾으먼되는 task로 object detection보다는 훨씬 간단한 task이다. → object의 크기, 위치, 경계, 클래스를 예측할 필요가 없다.

Experiments

Comparison to Other Real-Time Systems

Fast YOLO는 PASCAL에서 Real-Time Detector 중에서 가장 빠르고 정확한 object detection method이다.

Real-Time Detectors	Train	mAP	FPS
100Hz DPM [31]	2007	16.0	100
30Hz DPM [31]	2007	26.1	30
Fast YOLO	2007+2012	52.7	155
YOLO	2007+2012	63.4	45
Less Than Real-Time			
Fastest DPM [38]	2007	30.4	15
R-CNN Minus R [20]	2007	53.5	6
Fast R-CNN [14]	2007+2012	70.0	0.5
Faster R-CNN VGG-16[28]	2007+2012	73.2	7
Faster R-CNN ZF [28]	2007+2012	62.1	18
YOLO VGG-16	2007+2012	66.4	21

Table 1: Real-Time Systems on PASCAL VOC 2007. Comparing the performance and speed of fast detectors. Fast YOLO is the fastest detector on record for PASCAL VOC detection and is still twice as accurate as any other real-time detector. YOLO is 10 mAP more accurate than the fast version while still well above real-time in speed.

- YOLO의 backbone으로 VGG-16을 사용해 train해봤을 때, YOLO보다는 느렸지만 VGG-16을 사용하는 system과의 비교를 위해 진행했다.

VOC 2007 Error Analysis

YOLO와 Fast R-CNN을 조금 더 자세하게 비교 분석한다.

- Correct: correct class and $\text{IOU} > .5$
- Localization: correct class, $.1 < \text{IOU} < .5$
- Similar: class is similar, $\text{IOU} > .1$
- Other: class is wrong, $\text{IOU} > .1$
- Background: $\text{IOU} < .1$ for any object

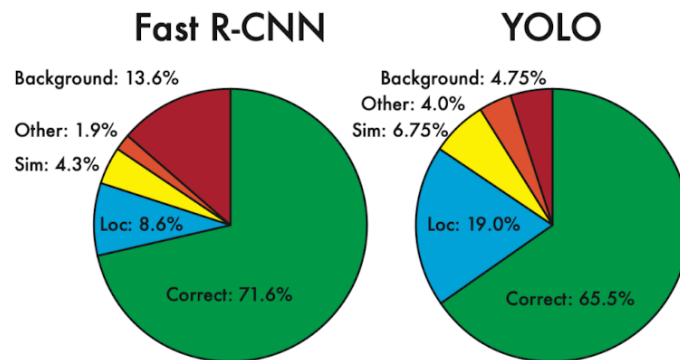


Figure 4: Error Analysis: Fast R-CNN vs. YOLO These charts show the percentage of localization and background errors in the top N detections for various categories ($N = \#$ objects in that category).

YOLO는 object를 옳게 localize하지는 못한다. YOLO의 localization error의 비율이 매우 높다.

Fast R-CNN은 localizaiton error를 훨씬 조금 만들지만 background error가 더 많다.

Combining Fast R-CNN and YOLO

Fast R-CNN과 YOLO를 boosting 방식으로 결합해 서로의 단점을 보완하도록 유도한다.

R-CNN이 예측한 모든 bounding box에 대해 YOLO도 비슷한 box를 예측했는지 확인하고, 그럴경우 prediction에 boost를 준다.

	mAP	Combined	Gain
Fast R-CNN	71.8	-	-
Fast R-CNN (2007 data)	66.9	72.4	.6
Fast R-CNN (VGG-M)	59.2	72.4	.6
Fast R-CNN (CaffeNet)	57.1	72.1	.3
YOLO	63.4	75.0	3.2

Table 2: Model combination experiments on VOC 2007. We examine the effect of combining various models with the best version of Fast R-CNN. Other versions of Fast R-CNN provide only a small benefit while YOLO provides a significant performance boost.

VOC 2012 Results

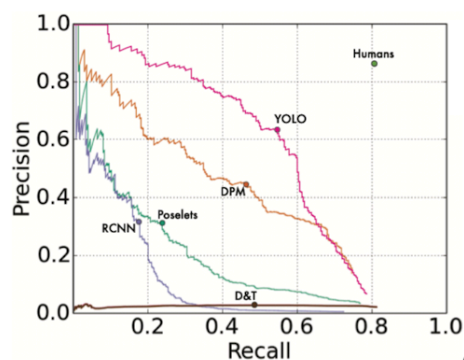
이 데이터셋에 대해서는 YOLO가 SOTA보다 낮은 VGG-16을 이용한 R-CNN과 비슷한 mAP을 보였다. 하지만 Fast R-CNN과 YOLO를 결합한 모델은 detection에서 매우 좋은 성능을 보였다.

VOC 2012 test	mAP	aero	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
MR.CNN.MORE.DATA [11]	73.9	85.5	82.9	76.6	57.8	62.7	79.4	77.2	86.6	55.0	79.1	62.2	87.0	83.4	84.7	78.9	45.3	73.4	65.8	80.3	74.0
HyperNet-VGG	71.4	84.2	78.5	73.6	55.6	53.7	78.7	79.8	87.7	49.6	74.9	52.1	86.0	81.7	83.3	81.8	48.6	73.5	59.4	79.9	65.7
HyperNet-SP	71.3	84.1	78.3	73.3	55.5	53.6	78.6	79.6	87.5	49.5	74.9	52.1	85.6	81.6	83.2	81.6	48.4	73.2	59.3	79.7	65.6
Fast R-CNN + YOLO	70.7	83.4	78.5	73.5	55.8	43.4	79.1	73.1	89.4	49.4	75.5	57.0	87.5	80.9	81.0	74.7	41.8	71.5	68.5	82.1	67.2
MR.CNN.S.CNN [11]	70.7	85.0	79.6	71.5	55.3	57.7	76.0	73.9	84.6	50.5	74.3	61.7	85.5	79.9	81.7	76.4	41.0	69.0	61.2	77.7	72.1
Faster R-CNN [28]	70.4	84.9	79.8	74.3	53.9	49.8	77.5	75.9	88.5	45.6	77.1	55.3	86.9	81.7	80.9	79.6	40.1	72.6	60.9	81.2	61.5
DEEP.ENS.COCCO	70.1	84.0	79.4	71.6	51.9	51.1	74.1	72.1	88.6	48.3	73.4	57.8	86.1	80.0	80.7	70.4	46.6	69.6	68.8	75.9	71.4
NoC [29]	68.8	82.8	79.0	71.6	52.3	53.7	74.1	69.0	84.9	46.9	74.3	53.1	85.0	81.3	79.5	72.2	38.9	72.4	59.5	76.7	68.1
Fast R-CNN [14]	68.4	82.3	78.4	70.8	52.3	38.7	77.8	71.6	89.3	44.2	73.0	55.0	87.5	80.5	80.8	72.0	35.1	68.3	65.7	80.4	64.2
UMICH.FGS.STRUCT	66.4	82.9	76.1	64.1	44.6	49.4	70.3	71.2	84.6	42.7	68.6	55.8	82.7	77.1	79.9	68.7	41.4	69.0	60.0	72.0	66.2
NUS.NIN.C2000 [7]	63.8	80.2	73.8	61.9	43.7	43.0	70.3	67.6	80.7	41.9	69.7	51.7	78.2	75.2	76.9	65.1	38.6	68.3	58.0	68.7	63.3
BabyLearning [7]	63.2	78.0	74.2	61.3	45.7	42.7	68.2	66.8	80.2	40.6	70.0	49.8	79.0	74.5	77.9	64.0	35.3	67.9	55.7	68.7	62.6
NUS.NIN	62.4	77.9	73.1	62.6	39.5	43.3	69.1	66.4	78.9	39.1	68.1	50.0	77.2	71.3	76.1	64.7	38.4	66.9	56.2	66.9	62.7
R-CNN VGG BB [13]	62.4	79.6	72.7	61.9	41.2	41.9	65.9	66.4	84.6	38.5	67.2	46.7	82.0	74.8	76.0	65.2	35.6	65.4	54.2	67.4	60.3
R-CNN VGG [13]	59.2	76.8	70.9	56.6	37.5	36.9	62.9	63.6	81.1	35.7	64.3	43.9	80.4	71.6	74.0	60.0	30.8	63.4	52.0	63.5	58.7
YOLO	57.9	77.0	67.2	57.7	38.3	22.7	68.3	55.9	81.4	36.2	60.8	48.5	77.2	72.3	71.3	63.5	28.9	52.2	54.8	73.9	50.8
Feature Edit [33]	56.3	74.6	69.1	54.4	39.1	33.1	65.2	62.7	69.7	30.8	56.0	44.6	70.0	64.4	71.1	60.2	33.3	61.3	46.4	61.7	57.8
R-CNN BB [13]	53.3	71.8	65.8	52.0	34.1	32.6	59.6	60.0	69.8	27.6	52.0	41.7	69.6	61.3	68.3	57.8	29.6	57.8	40.9	59.3	54.1
SDS [16]	50.7	69.7	58.4	48.5	28.3	28.8	61.3	57.5	70.8	24.1	50.7	35.9	64.9	59.1	65.8	57.1	26.0	58.8	38.6	58.9	50.7
R-CNN [13]	49.6	68.1	63.8	46.1	29.4	27.9	56.6	57.0	65.9	26.5	48.7	39.5	66.2	57.3	65.4	53.2	26.2	54.5	38.1	50.6	51.6

Table 3: PASCAL VOC 2012 Leaderboard. YOLO compared with the full comp4 (outside data allowed) public leaderboard as of November 6th, 2015. Mean average precision and per-class average precision are shown for a variety of detection methods. YOLO is the only real-time detector. Fast R-CNN + YOLO is the forth highest scoring method, with a 2.3% boost over Fast R-CNN.

Generalizability: Person Detection in Artwork

도메인 일반화 능력을 테스트하기 위해 예술작품에서 사람을 탐지하는 능력을 평가한다.



	VOC 2007 AP	Picasso AP	Best F_1	People-Art AP
YOLO	59.2	53.3	0.590	45
R-CNN	54.2	10.4	0.226	26
DPM	43.2	37.8	0.458	32
Poselets [2]	36.5	17.8	0.271	
D&T [4]	-	1.9	0.051	

(b) Quantitative results on the VOC 2007, Picasso, and People-Art Datasets. The Picasso Dataset evaluates on both AP and best F_1 score.

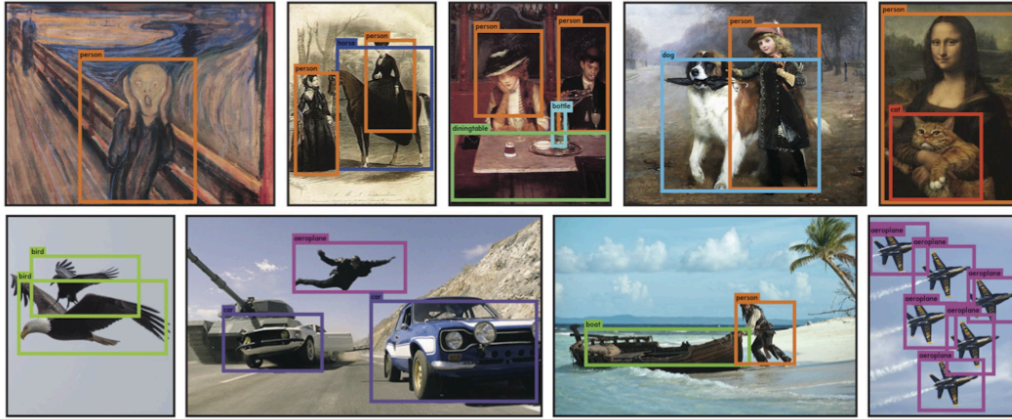


Figure 6: Qualitative Results. YOLO running on sample artwork and natural images from the internet. It is mostly accurate although it does think one person is an airplane.

- R-CNN은 VOC 2007에서 매우 좋은 성능을 보이지만 artwork에 대해서는 성능이 상당히 떨어진다. R-CNN은 region proposal로 selective search 알고리즘을 사용하는데 이는 natural image에 특화되어있다.
- DPM은 artwork에서도 성능을 비슷하게 유지하였는데 이는 object의 shape과 layout에 대한 좋은 spatial model을 가지고있기 때문이다.
- YOLO는 VOC 2007에서도 좋은 성능을 보인다.

DPM처럼 YOLO는 object의 크기와 형태, object들끼리의 연관성 등을 모델링한다.

Artwork와 natural image는 pixel level에서는 매우 다르지만 크기와 형태 면에서는 매우 비슷하기때문에 DPM과 YOLO 같은 모델의 성능이 떨어지지 않는 이유이다.

Real-Time Detection In The Wild

YOLO를 실제 webcam과 연결해 real-time performance에 대해 평가한다.

결과적으로 YOLO는 독립된 image에서 작동하지만 tracking system과 같은 역할을 한다.

Summary

해당 논문에서는 기존 2-stage로 수행하던 object detection을 1-stage로 수행할 수 있는 unified model YOLO를 제안한다. ⇒ end-to-end 학습이 가능

또한 가장 빠른 general-purpose object detection 모델인 Fast YOLO도 제안한다.

- Real-time object detection SOTA를 달성했다. ⇒ 매우 빠름

- 새로운 domain으로의 gernalize가 잘된다. ⇒ Generalize 성능 향상